AI Image Generator

Technical Documentation, Workflows and Requirements

Version 1.0
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Stable Diffusion XL + Real-ESRGAN, served locally with Gradio, with per-image GPU telemetry recorded in SQLite.

Contents

1. Project Overview 4

1.1 Purpose . . . 4

1.2 Key capabilities . . . 4

1.3 Target environment . . . 4

1.4 Document map . . . 5

2. Requirements 6

2.1 Functional requirements . . . 6

2.2 Non-functional requirements . . . 6

2.3 Hardware requirements . . . 6

2.4 Software requirements . . . 7

2.5 Model assets . . . 7

2.6 Constraints and assumptions . . . 7

3. System Architecture 8

3.1 Component overview . . . 8

3.2 Generation pipeline . . . 8

3.3 Data flow and storage . . . 9

3.4 Design decisions . . . 9

3.5 Concurrency model . . . 9

4. Workflows 10

4.1 First-time setup . . . 10

4.2 Daily use: generating an image . . . 10

4.3 Producing 4K and 8K output . . . 11

4.4 Reviewing past generations . . . 11

4.5 Batch generation from the command line . . . 12

4.6 Upscaling an existing image . . . 12

4.7 Stopping and restarting . . . 12

4.8 Backup . . . 12

4.9 Development workflow . . . 12

5. Module Reference 13

5.1 app.py (355 lines) . . . 13

5.2 prompt_parser.py (252 lines) . . . 13

5.3 upscaler.py (109 lines) . . . 14

5.4 gpu_info.py (97 lines) . . . 14

5.5 db.py (177 lines) . . . 14

5.6 generate.py (108 lines) . . . 14

5.7 Supporting files . . . 15

6. Prompt Language Reference **16**

6.1 Size and shape . . . 16

6.2 Output resolution (upscaling) . . . 16

6.3 Quality and fidelity . . . 17

6.4 Style presets . . . 17

6.5 Batch and reproducibility . . . 17

6.6 Worked examples . . . 17

7. Database **18**

7.1 Overview . . . 18

7.2 Table: generations . . . 18

7.3 Example queries . . . 19

7.4 Access from Python . . . 20

8. Performance **21**

8.1 Generation (native size) . . . 21

8.2 Upscaling . . . 21

8.3 End-to-end . . . 21

8.4 Memory . . . 21

8.5 Startup . . . 21

8.6 Aggregate over the session . . . 22

8.7 Tuning levers . . . 22

9. Operations and Troubleshooting **23**

9.1 Starting and stopping . . . 23

9.2 Logs . . . 23

9.3 Known issues encountered during development and their fixes . . . 23

9.4 Troubleshooting guide . . . 24

9.5 Maintenance . . . 24

9.6 Security notes . . . 24

10. Change Log and Roadmap **25**

10.1 Change log . . . 25

10.2 Roadmap . . . 26

Appendix **27**

A. Project file tree . . . 27

B. requirements.txt . . . 27

C. Command cheat sheet . . . 27

D. Programmatic access to the running app . . . 28

E. Licences . . . 28

1. Project Overview

1.1 Purpose

AI Image Generator is a self-hosted application that turns a plain-English prompt into an image using the Stable Diffusion XL model, running entirely on a local NVIDIA GPU. No image data leaves the machine and there is no per-image cost.

The project was built in a single development session on 21 and 22 October 2026 and grew through five increments:

1. A command-line script that runs Stable Diffusion XL on the GPU.
2. A local web interface so a prompt can be typed and the result viewed in a browser.
3. A prompt language that replaces sliders and dropdowns: size, ratio, quality, style, count and seed are all read from the prompt text.
4. Per-image telemetry (timings, VRAM, temperature, power, clocks) shown in the UI and stored in a SQLite database with a History tab.
5. A Real-ESRGAN upscaling stage so that "4k" or "8k" in the prompt produces a genuine 3840 or 7680 pixel image.

1.2 Key capabilities

- Text-to-image generation with Stable Diffusion XL base 1.0 in 16-bit precision.
- Prompt-driven control of aspect ratio, exact resolution, orientation, quality, fidelity, art style, image count, seed, steps and guidance.
- Live interpretation line under the prompt box showing exactly what will be generated before the button is pressed.
- Tiled Real-ESRGAN upscaling to 2K, 4K, 8K or any exact size, with automatic centre crop.
- Full telemetry per image: inference time, steps per second, peak VRAM, GPU temperature and power before and after, clocks, fan, performance state, driver and library versions.
- Persistent history in SQLite with search, aggregate statistics, and click-to-view of any past image with its full record.
- Command-line tools for batch generation, upscaling an existing image, and querying the database.

1.3 Target environment

The application was developed and verified on the following machine. Other NVIDIA GPUs with at least 12 GB of VRAM will work; see the Requirements section for the full matrix.

Item	Value
GPU	NVIDIA GeForce RTX 3090, 24 GB VRAM, compute capability 8.6, 82 SMs
Driver	610.88
OS	Windows 11 Pro 10.0.26300
Python	3.12.0
CUDA runtime (via PyTorch)	12.6
Project folder	C:\Users\frazi\Documents\apps\ai-image-gen

1.4 Document map

Section	Contents
2. Requirements	Functional and non-functional requirements, hardware and software matrix
3. Architecture	Components, pipeline diagram, data flow, design decisions
4. Workflows	Setup, generation, upscaling, history and maintenance workflows step by step
5. Module reference	Every source file, its public functions and their contracts
6. Prompt language	Complete reference of words the parser understands
7. Database	Schema of generations.db, every column explained, example queries
8. Performance	Measured timings and resource use on the RTX 3090
9. Operations	Running, stopping, troubleshooting, known issues, backup
10. Change log and roadmap	What was built in which order, defects fixed, future work
Appendix	File tree, dependency list, command cheat sheet

2. Requirements

2.1 Functional requirements

ID	Requirement	Status
FR-01	Generate an image from a text prompt using a locally hosted diffusion model on the GPU.	Done
FR-02	Provide a browser interface where a prompt can be typed and the result is displayed.	Done
FR-03	Keep the model loaded between requests so repeat generations take seconds, not minutes.	Done
FR-04	Control aspect ratio, resolution, orientation, quality, fidelity, style, count and seed from words in the prompt, with no separate controls.	Done
FR-05	Show how the prompt was interpreted before generation starts.	Done
FR-06	Produce genuine 4K (3840 px) and 8K (7680 px) output when requested, and exact sizes such as 3840x2160.	Done
FR-07	Display, for every image, the GPU used, time taken, VRAM, temperature, power, clocks and software versions.	Done
FR-08	Persist every generation with its settings and telemetry in a database that can be browsed later.	Done
FR-09	Provide a History view with search, aggregate statistics and click-to-view of past images.	Done
FR-10	Save every generated image, native and upscaled, as a PNG file on disk.	Done
FR-11	Support a negative prompt describing what to avoid.	Done
FR-12	Provide command-line equivalents for generation, upscaling and database queries.	Done

2.2 Non-functional requirements

ID	Requirement	How it is met
NFR-01	Runs fully offline after first model download.	Model weights cached in the Hugging Face cache; upscaler weights stored in models/.
NFR-02	A 1 megapixel image generates in under 15 seconds on an RTX 3090.	Measured 4.7 to 13.5 s depending on steps and size.
NFR-03	Peak VRAM stays under 12 GB so 12 GB and 16 GB cards can run it.	Measured peak 10.5 GB including the upscaler.
NFR-04	Upscaling must never exceed VRAM regardless of output size.	512 px tiles with 32 px overlap; memory is bounded per tile.
NFR-05	The server binds to localhost only.	Gradio launched on 127.0.0.1:7860.
NFR-06	All persistent state is in plain files that can be backed up by copying.	PNG files in outputs/, SQLite file generations.db.
NFR-07	The database schema can evolve without losing data.	db.connect() adds missing columns automatically.
NFR-08	Code is dependency-light and readable: no framework beyond Gradio and diffusers.	Six Python modules, about 1,100 lines in total.

2.3 Hardware requirements

Component	Minimum	Recommended	Verified on
GPU	NVIDIA, 12 GB VRAM, compute capability 7.0+	16 GB or more	RTX 3090, 24 GB
System RAM	16 GB	32 GB	-
Disk	15 GB free (7 GB model, 64 MB upscaler, Python environment, outputs)	50 GB for a large output library	294 GB free
CPU	Any x86-64	-	-

Cards below 12 GB can run the command-line script, which enables CPU offload automatically, but the web app assumes the full pipeline fits in VRAM.

2.4 Software requirements

Package	Version used	Purpose
Python	3.12.0	Runtime
torch	2.14.1+cu126	Tensor library with CUDA 12.6
torchvision	0.29.1+cu126	Required by transformers image processors and spandrel
diffusers	0.41.0	Stable Diffusion XL pipeline
transformers	5.20.2	CLIP text encoders used by SDXL
accelerate	1.15.0	Device placement helpers
safetensors	0.8.0	Safe weight loading
gradio	6.30.0	Web interface
gradio_client	2.7.2	Used in tests to call the app programmatically
spandrel	0.4.2	Loads the Real-ESRGAN architecture and weights
pandas	3.0.6	History table in the UI
Pillow	12.3.0	Image I/O, resizing, cropping
numpy	2.5.4	Tile arrays for the upscaler
NVIDIA driver	610.88	Also provides nvidia-smi for telemetry

2.5 Model assets

Asset	Source	Size	Location
stabilityai/stable-diffusion-xl-base-1.0 (fp16 safetensors)	Hugging Face Hub	6.7 GB	%USERPROFILE%\.cache\huggingface\hub
RealESRGAN_x4plus.pth	github.com/xinntao/Real-ESRGAN releases v0.1.0	64 MB	models/ inside the project

2.6 Constraints and assumptions

- Stable Diffusion XL is trained at about 1 megapixel. Native generation is therefore capped at a 1536 px long side; larger sizes are reached by upscaling.
- The telemetry module depends on nvidia-smi being on the PATH, which the NVIDIA driver installer guarantees on Windows. If it is missing, telemetry fields are stored as NULL and generation still works.
- The application is single-user and single-GPU. Concurrent requests are queued by Gradio and processed one at a time.
- The SDXL licence (CreativeML Open RAIL++-M) and the Real-ESRGAN licence (BSD-3) permit local and commercial use with the usual use restrictions; the operator is responsible for the content they generate.

3. System Architecture

3.1 Component overview

The application is a single Python process. Gradio serves the browser interface and calls plain Python functions; those functions use five project modules and two GPU-resident models.

Component	File	Responsibility
Web application	app.py	Builds the Gradio UI, owns the loaded models, orchestrates one generation end to end, renders the details report, serves the History tab.
Prompt parser	prompt_parser.py	Turns free text into a Settings object: size, steps, guidance, count, seed, style, upscale target. Pure function, no GPU.
Upscaler	upscaler.py	Wraps Real-ESRGAN through spandrel; tiled x4 passes; resizes or crops to the requested target.
GPU telemetry	gpu_info.py	Static GPU facts from torch; live temperature, power, clocks, fan and load from nvidia-smi.
Database	db.py	SQLite schema, migration, insert, fetch, search, per-row get, aggregate stats, command-line viewer.
CLI generator	generate.py	Standalone batch generation without the web UI; the original script from increment 1.

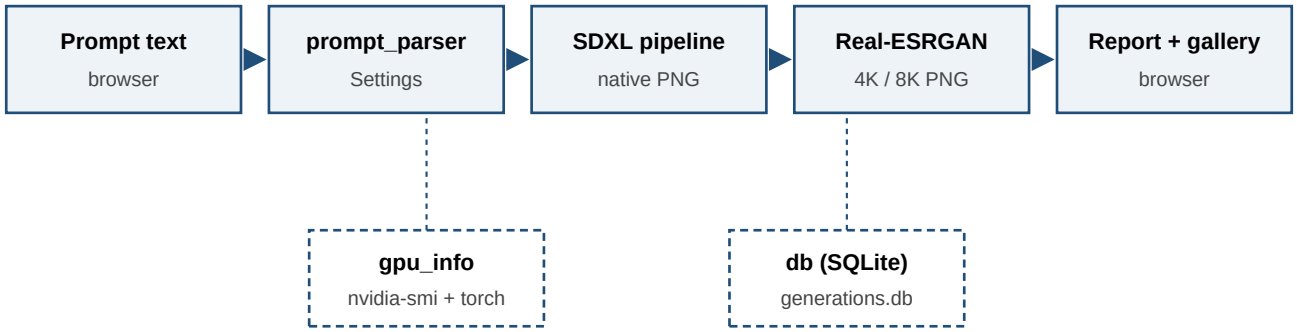


Figure 3.1: Generation pipeline. Solid boxes are the per-image data path; dashed boxes are sampled or written around steps 3 and 4.

3.2 Generation pipeline

For every image the sequence is:

1. The prompt text is parsed into a Settings object. Numeric directives are stripped; style words add descriptor suffixes.
2. The seed is fixed (from the prompt or random) and a CUDA generator is created.
3. Peak-memory counters are reset and a telemetry snapshot is taken (temperature, power, clocks).
4. The SDXL pipeline runs for the requested number of steps at the native size. A per-step callback drives the progress bar.
5. CUDA is synchronised, the inference time is recorded, and a second telemetry snapshot is taken while the GPU is still under load.
6. The native image is saved as PNG.
7. If an upscale target is set, the Real-ESRGAN stage runs: one or two tiled x4 passes, then a Lanczos resample to the target, then an optional centre crop to an exact size. The upscaled PNG is saved.
8. A record with about 50 fields is inserted into generations.db.

9. The final image (upscaled if applicable) and a Markdown report are returned to the browser.

3.3 Data flow and storage

Data	Where it lives	Format	Written by
Native images	outputs/YYYYMMDD-HHMMSS_seedN.png	PNG, 1 to 2 MB	app.py, generate.py
Upscaled images	outputs/..._seedN_3840px.png	PNG, 8 to 31 MB	app.py
Generation records	generations.db	SQLite, one table	db.py
Model weights	Hugging Face cache, models/	safetensors, pth	downloaded once
Logs	stdout of the app process	text	app.py

3.4 Design decisions

Prompt as the only control surface. Early versions had dropdowns for style and size and sliders for steps and guidance. These were removed in favour of a parser because a single text box is faster to use, is self-documenting through the live interpretation line, and makes every generation reproducible from the prompt string alone, which is stored in the database.

Two-stage resolution. SDXL produces incoherent output above roughly 1 megapixel and attention cost grows with the square of pixel count. Generating at native size and upscaling with a super-resolution network is the standard approach, gives better results, and completes in a few seconds. The parser treats "4k" and "8k" as output-size requests, not as native resolutions.

Tiled upscaling. Real-ESRGAN on a full 3840 px image would need tens of gigabytes for intermediate feature maps. Processing 512 px tiles with a 32 px overlap bounds memory at a few hundred megabytes and hides seams.

Telemetry via nvidia-smi. Shelling out to nvidia-smi avoids a dependency on NVML Python bindings, works on every driver, and costs about 100 ms per call. Two calls per image is negligible against a 10 s generation.

SQLite over a server database. A single-user desktop tool needs no server. SQLite is in the standard library, the file is trivially backed up, and any SQL client can open it. The connect() helper adds missing columns so the schema can grow without migrations scripts.

Models stay resident. Loading SDXL takes about 30 s. Both models are loaded once at process start and held on the GPU for the lifetime of the server, so each generation pays only inference cost.

3.5 Concurrency model

Gradio runs the generate function in a worker thread and queues additional requests. The GPU is used by one generation at a time. The History tab reads the database on its own thread; SQLite handles the concurrent read safely because each call opens its own short-lived connection.

4. Workflows

4.1 First-time setup

1. Confirm the GPU and driver are visible: run `nvidia-smi` in a terminal. It must list the card and a driver version.
2. Open PowerShell in the project folder.
3. Create and activate a virtual environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

4. Install the CUDA build of PyTorch, then the remaining dependencies:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt
```

5. Verify CUDA is active:

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

6. Confirm the upscaler weights exist at models\RealESRGAN_x4plus.pth. If not, download them from the Real-ESRGAN v0.1.0 release page into that folder.
7. Start the app once. The first start downloads the 6.7 GB SDXL model into the Hugging Face cache and can take several minutes depending on bandwidth.

4.2 Daily use: generating an image

1. Double-click run-app.bat, or run `.\.venv\Scripts\python.exe app.py` from the project folder.
2. Wait about 30 seconds for "Model ready" and "Upscaler ready" in the console. The browser opens http://127.0.0.1:7860 automatically.
3. Type a prompt. As you type, the line under the box shows the interpreted settings, for example "1344x768 | upscale to 3840x2194 | 40 steps | guidance 7".
4. Optionally edit the negative prompt.
5. Press Generate or Enter. A progress bar shows the step count and, if applicable, the upscale stage.
6. The image appears in the gallery. The details panel below shows timing, settings, GPU telemetry and the exact prompt sent to the model.
7. Both files are already saved in outputs\. Nothing further is required.



Figure 4.1: "a lighthouse on a rocky cliff at golden hour, cinematic, 16:9, 4k, seed 42" rendered at 1344x768 and upscaled to 3840x2194.

4.3 Producing 4K and 8K output

1. Include `4k`, `8k`, `2k` or an exact size such as `3840x2160` in the prompt.
2. The parser sets an upscale target and raises steps to 40 for a cleaner base.
3. Generation runs at the native bucket for the aspect ratio (for 16:9 that is 1344x768).
4. The upscaler runs one x4 pass for 4K, or two passes for 8K with an intermediate resize so the second pass lands on the target.
5. If an exact size was given, the result is centre-cropped to it.
6. The gallery and History show the upscaled image; the native image is also kept on disk.

Recommended practice: iterate at native size until the composition is right, then add `4k` and the same `seed N` to the prompt to upscale that exact image.

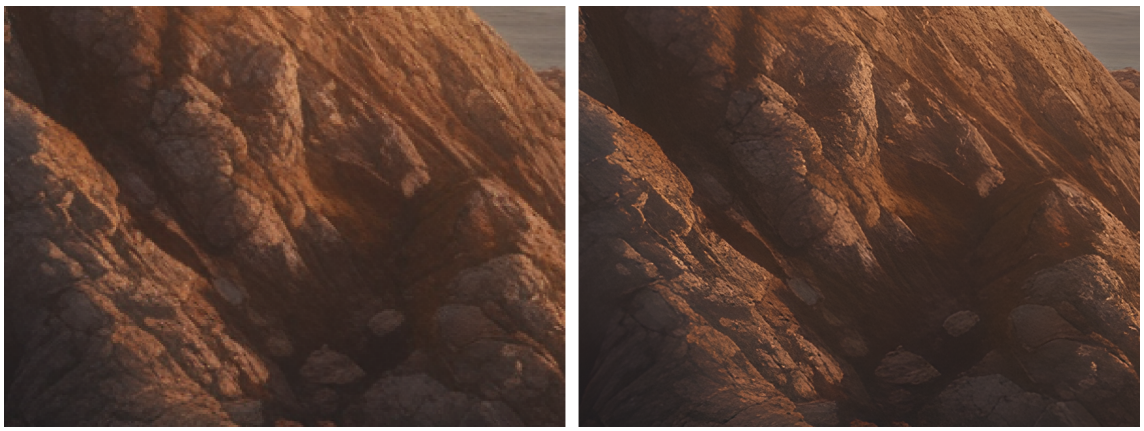


Figure 4.2: The same 300x225 region of the lighthouse cliff. Left: native pixels enlarged. Right: Real-ESRGAN output. The upscaler reconstructs rock texture rather than blurring.

4.4 Reviewing past generations

1. Open the History tab. The top line shows totals: number of images, GPU seconds, average and extreme timings, VRAM, hottest temperature and peak power.
2. Type part of a prompt in the search box and press Enter to filter, or press Refresh to reload.
3. Click a row. The image (upscaled if one exists) and its full details appear below the table.
4. For scripted access, run from the project folder:

```
.\.venv\Scripts\python.exe db.py          # last 20 rows
.\.venv\Scripts\python.exe db.py 12       # every field of row 12
.\.venv\Scripts\python.exe db.py stats    # aggregates
```

4.5 Batch generation from the command line

The original script is retained for scripted use. It loads the model per run, so it is best for batches:

```
.\.venv\Scripts\python.exe generate.py "a red sports car" --steps 40 --seed 42 --count 4
.\.venv\Scripts\python.exe generate.py --help
```

It writes to outputs\ but does not record to the database or upscale.

4.6 Upscaling an existing image

```
.\.venv\Scripts\python.exe upscaler.py outputs\some_image.png 3840
```

The result is written next to the input with an `_up3840` suffix.

4.7 Stopping and restarting

- Close the console window, or press Ctrl+C in it, to stop the server. Loaded models are released.
- Restart with run-app.bat. Startup is about 30 to 45 seconds because the models are reloaded from disk.
- If port 7860 is reported busy, a previous instance is still running; end it from Task Manager (python.exe) or change the port in the last line of app.py.

4.8 Backup

Copy two things: the outputs\ folder and generations.db. The models can be re-downloaded and the virtual environment can be rebuilt from requirements.txt.

4.9 Development workflow

1. Edit a module. The parser and database have self-tests: `python prompt_parser.py` prints how sample prompts are interpreted, and `python db.py stats` exercises the schema.
2. Check syntax quickly with `python -W error -c "import ast; ast.parse(open('app.py', encoding='utf-8').read())"`.
3. Restart the app. For automated checks, gradio_client can call the app's `/generate` and `/refresh_history` endpoints, which is how the increments in this project were verified.

5. Module Reference

5.1 app.py (355 lines)

Entry point of the web application.

Name	Kind	Description
MODEL_ID	constant	"stabilityai/stable-diffusion-xl-base-1.0"
OUTPUT_DIR	constant	Path to outputs/, created on import
DEFAULT_NEGATIVE	constant	"blurry, low quality, deformed, watermark, text"
HELP	constant	Markdown table of prompt words shown in the collapsible help panel
load_pipeline()	function	Loads SDXL in float16 with the fp16 variant, moves it to CUDA, enables VAE slicing, disables the console progress bar. Exits if CUDA is unavailable. Returns the pipeline.
PIPE, UPSCALER, STATIC_GPU	globals	The loaded pipeline, the Upscaler instance, and static GPU facts, created once at import.
preview(prompt)	function	Returns the Markdown "Will generate:" line for the live interpretation display.
details_markdown(rec)	function	Renders one database record as the Markdown report shown under the gallery and in History.
generate(prompt, negative, progress)	function	Runs the full pipeline for all requested images. Returns (list of PIL images, Markdown report). Raises gr.Error on an empty prompt.
history_table(search, limit)	function	Returns a pandas DataFrame with the columns shown in the History grid.
history_stats()	function	Returns the one-line aggregate summary.
refresh_history(search, limit)	function	Event handler returning (table, stats).
select_history(evt, table)	function	Row-click handler returning (image path, details Markdown). Prefers the upscaled file.

The UI is built in a gr.Blocks context with two tabs. Events wired: prompt change to preview, button click and Enter to generate, History tab select and Refresh click and search submit to refresh_history, table row select to select_history.

5.2 prompt_parser.py (252 lines)

Pure parsing, no GPU or I/O.

Name	Kind	Description
Settings	dataclass	prompt, width, height, steps, guidance, count, seed, style, upscale_to, exact_size, notes. Methods: summary() for display, upscaled_size() for the final output size.
RATIO_BUCKETS	constant	Eleven SDXL-friendly (width, height) pairs keyed by ratio string.
ORIENTATION_WORDS	constant	Ordered regex to ratio mapping; first match wins.
STYLE_SUFFIXES	constant	Ordered regex to descriptor list; first match wins; descriptors already in the prompt are not duplicated.
UPSCALE_TARGETS	constant	2k 2560, 4k 3840, uhd 3840, 8k 7680 (long side).
parse(text)	function	Returns a Settings. Order of evaluation: explicit resolution, explicit ratio, orientation word, upscale words, steps, guidance, count, seed, style, tidy-up.

Internal helpers: _take() finds a pattern in the current prompt and removes it (case-insensitive) so later patterns see the updated text; _fit_resolution() scales a requested size to about 1 megapixel and snaps to multiples of 64 within 512 to 1536; _closest_bucket() picks the nearest ratio for unusual ratios.

Running the module directly prints the interpretation of nine sample prompts.

5.3 upscaler.py (109 lines)

Name	Kind	Description
MODEL_PATH	constant	models/RealESRGAN_x4plus.pth
TILE, OVERLAP	constants	512 and 32 pixels
Upscaler(device, model_path)	class	Loads the model with spandrel, switches it to half precision on CUDA, exposes name and scale.
Upscaler.upscale_once(img)	method	One x4 pass. Iterates tiles with context padding, runs each through the network under inference_mode, crops the padding, writes into the output array.
Upscaler.upscale_to(img, long_side)	method	Repeats passes until the long side reaches the target; before a second pass resizes the intermediate so the pass lands near the target; finally Lanczos-resamples down to exactly long_side. Returns (image, info dict with upscaler, passes, from, to, seconds).

Running the module directly upscales a file given on the command line.

5.4 gpu_info.py (97 lines)

Name	Kind	Description
snapshot()	function	Runs nvidia-smi once with a fixed query and returns a dict: temperature_c, power_w, power_limit_w, gpu_util_pct, mem_util_pct, mem_used_mb, mem_total_mb, sm_clock_mhz, mem_clock_mhz, fan_pct, driver_version, pstate. Returns {} if nvidia-smi fails.
static_info()	function	Returns gpu_name, vram_total_gb, compute_capability, multiprocessors, cuda_version, cudnn_version, torch_version, diffusers_version, python_version, os.
torch_memory()	function	Returns vram_allocated_gb, vram_reserved_gb, vram_peak_gb from the torch allocator.

5.5 db.py (177 lines)

Name	Kind	Description
DB_PATH	constant	generations.db next to the module
SCHEMA	constant	CREATE TABLE IF NOT EXISTS for generations plus an index on created_at
NEW_COLUMNS	constant	Columns added after the first release; applied by ALTER TABLE when missing
connect()	function	Opens a connection with Row factory, applies schema and migrations, returns it
insert(record)	function	Inserts known keys as columns; unknown keys are packed into extra_json. Returns the row id
fetch(limit, search)	function	Newest-first list of dicts, optionally filtered by LIKE on user_prompt or final_prompt
get(row_id)	function	One record as a dict, or None
stats()	function	Aggregates: count, total and average times, fastest and slowest, average steps per second, VRAM averages and maxima, hottest temperature, peak power, upscaled count and time, first and last timestamps

Running the module directly provides the command-line viewer described in the Workflows section.

5.6 generate.py (108 lines)

Standalone command-line generator. Arguments: prompt, --negative, --steps, --guidance, --width, --height, --seed, --count, --model, --out. Loads the pipeline per invocation, enables CPU offload on cards under 12 GB, saves PNGs with timestamp and seed, and prints peak VRAM at the end. It does not write to the database.

5.7 Supporting files

File	Purpose
requirements.txt	Python dependencies with the note on installing the CUDA PyTorch build first
run-app.bat	Double-click launcher that runs app.py inside the virtual environment
README.md	User-facing quick start and prompt reference
models/RealESRGAN_x4plus.pth	Upscaler weights
docs/	This documentation set as PDFs; docs-src/ holds the Markdown sources, images and build script

6. Prompt Language Reference

Everything about a generation is expressed in the prompt. Matching is case-insensitive. Numeric directives are removed before the prompt reaches the model; descriptive words stay in because they also guide the image.

6.1 Size and shape

Write	Effect	Generated at
16:9	Widescreen	1344x768
9:16	Vertical widescreen	768x1344
4:3 / 3:4	Classic	1152x896 / 896x1152
3:2 / 2:3	Photo	1216x832 / 832x1216
1:1	Square	1024x1024
21:9 / 9:21	Ultrawide	1536x640 / 640x1536
5:4 / 4:5	Print	1088x896 / 896x1088
1920x1080 (any WxH)	Same ratio, scaled to about 1 megapixel; if larger than native, the output is upscaled and cropped to exactly that size	depends
portrait, tall, vertical, poster, book cover	2:3	832x1216
landscape, wide, horizontal	3:2	1216x832
square, icon, avatar, profile picture	1:1	1024x1024
wallpaper, desktop wallpaper, widescreen, banner, youtube thumbnail	16:9	1344x768
phone wallpaper, mobile wallpaper, instagram story, tiktok	9:16	768x1344
ultrawide, cinemascope	21:9	1536x640

Precedence: an explicit WxH beats an explicit ratio, which beats orientation words. Among orientation words the first match in the table order wins, so "portrait of a king, landscape" is tall.

6.2 Output resolution (upscaling)

Write	Long side after upscaling	Passes
2k	2560 px	1
4k or uhd	3840 px	1
8k	7680 px	2
3840x2160 (any size above native)	exactly that size, centre-cropped	1 or 2

Any of these also raises steps to 40 unless steps are given explicitly.

6.3 Quality and fidelity

Write	Effect
hd, ultra detailed, highly detailed, masterpiece, best quality, high quality	40 steps
quick, draft, fast, rough, preview	15 steps
25 steps, steps 25, steps=25	exactly that (1 to 100)
strict, exact, precise, literal	guidance 9 (follows the prompt closely)
creative, loose, dreamy, abstract, surreal	guidance 5 (more freedom)
cfg 8, guidance 8	exactly that (1 to 20)

Default is 30 steps and guidance 7.

6.4 Style presets

The first matching style adds descriptors that are not already present.

Write	Descriptors added
photorealistic, photo, photograph, dslr	photorealistic, 8k, DSLR photo, sharp focus, natural lighting
anime, manga	anime style, vibrant colors, clean lines, studio quality
cinematic, film still, movie scene	cinematic lighting, dramatic, film still, depth of field, color graded
watercolor	watercolor painting, soft edges, paper texture, pastel tones
oil painting, oil on canvas	oil painting, visible brush strokes, canvas texture, classical
3d render, 3d, octane, blender, unreal	3d render, octane render, volumetric lighting, highly detailed
pixel art, 8-bit, 16-bit	pixel art, retro game sprite, crisp pixels
digital art, concept art, digital painting	digital painting, concept art, trending on artstation, highly detailed
sketch, pencil sketch, line art, ink drawing	pencil sketch, line art, monochrome, hand drawn
comic, cartoon	comic book style, bold outlines, flat colors, halftone
cyberpunk	cyberpunk, neon lights, rain, futuristic city, high contrast
low poly	low poly, geometric, isometric, minimal shading

Note that the photorealistic preset contains the word "8k" as a quality descriptor; it does not trigger 8K upscaling because upscale words are detected on the original prompt before the suffix is added.

6.5 Batch and reproducibility

Write	Effect
3 images, 4 variations, 2 versions, x4	1 to 4 images, seeds N, N+1, N+2 ...
seed 42, seed: 42, seed=42	fixed seed; omitted means random

6.6 Worked examples

Prompt	Interpreted as
a cozy cabin in a snowy forest, 16:9, 4k	1344x768, upscale to 3840x2194, 40 steps, guidance 7
portrait of a cyberpunk samurai, 3 images, seed 42	832x1216, 30 steps, style cyberpunk, 3 images, seeds 42 to 44
desktop wallpaper of a mountain lake, photorealistic, 8k	1344x768, upscale to 7680x4389, 40 steps, style photorealistic
a cat, 1920x1080, quick	1344x768, upscale and crop to 1920x1080, 15 steps
oil painting of a ship in a storm 21:9 steps 45 cfg 8	1536x640, 45 steps, guidance 8, style oil painting
square icon of a fox, 2k	1024x1024, upscale to 2560x2560, 40 steps

7. Database

7.1 Overview

File: generations.db in the project folder. Engine: SQLite 3 through Python's standard library. One table, generations, one row per image produced by the web app. An index on created_at supports newest-first listing.

The schema is created on first connection. Columns added in later increments are listed in NEW_COLUMNS and added with ALTER TABLE when absent, so an older database file is upgraded in place the first time the new code opens it.

7.2 Table: generations

Identity and files

Column	Type	Meaning
id	INTEGER PK	Auto-increment row id, shown as "#N" in the UI
created_at	TEXT	ISO-8601 local timestamp, seconds precision
image_path	TEXT	Absolute path of the native PNG
file_size_kb	REAL	Size of the native PNG

Prompt

Column	Type	Meaning
user_prompt	TEXT	Exactly what was typed
final_prompt	TEXT	What was sent to the model after directive removal and style suffixes
negative_prompt	TEXT	Negative prompt used
parser_notes	TEXT	Human-readable list of parser decisions, joined with " - "

Settings

Column	Type	Meaning
model_id	TEXT	Hugging Face model id
width, height	INTEGER	Native generation size
steps	INTEGER	Denoising steps
guidance	REAL	Classifier-free guidance scale
seed	INTEGER	Seed used for this image
style	TEXT	Name of the matched style preset, or NULL
batch_index, batch_size	INTEGER	Position within a multi-image request

Timing

Column	Type	Meaning
inference_seconds	REAL	Pipeline call, from start to CUDA synchronise
save_seconds	REAL	PNG encode and write of the native image
total_seconds	REAL	inference + save (excludes upscaling)
steps_per_second	REAL	steps / inference_seconds

GPU and software

Column	Type	Meaning
gpu_name	TEXT	From torch device properties
vram_total_gb	REAL	Card memory
vram_peak_gb	REAL	Peak torch allocation during this image, including upscaling
vram_reserved_gb	REAL	Allocator reservation after the image
compute_capability	TEXT	e.g. "8.6"
driver_version	TEXT	From nvidia-smi
cuda_version, torch_version, diffusers_version	TEXT	Library versions

Telemetry (from nvidia-smi)

Column	Type	Meaning
temp_before_c, temp_after_c	REAL	GPU temperature before the pipeline call and right after it
power_before_w, power_after_w	REAL	Board power draw at the same two moments
power_limit_w	REAL	Configured power limit
gpu_util_after_pct	REAL	GPU utilisation at the after snapshot
sm_clock_mhz, mem_clock_mhz	REAL	Clock speeds at the after snapshot
fan_after_pct	REAL	Fan speed
pstate	TEXT	Performance state, P0 to P8

Upscaling (added in increment 5)

Column	Type	Meaning
upscale_target	INTEGER	Requested long side in pixels
upscaled_width, upscaled_height	INTEGER	Final size after resample and crop
upscaled_path	TEXT	Absolute path of the upscaled PNG
upscaled_size_kb	REAL	Size of that file
upscale_seconds	REAL	Time of the upscaling stage
upscale_passes	INTEGER	Number of x4 passes (1 or 2)
upscaler	TEXT	Architecture and weight file name

Catch-all

Column	Type	Meaning
extra_json	TEXT	JSON of any record keys that have no column, for forward compatibility

7.3 Example queries

Average inference time by native size:

```
SELECT width || 'x' || height AS size, COUNT(*) AS n,
       ROUND(AVG(inference_seconds), 2) AS avg_s
FROM generations GROUP BY size ORDER BY n DESC;
```

Hottest runs:

```
SELECT id, temp_after_c, power_after_w, user_prompt
FROM generations ORDER BY temp_after_c DESC LIMIT 5;
```

Every 4K or larger image with its file:

```
SELECT id, upscaled_width, upscaled_height, upscale_seconds, upscaled_path
FROM generations WHERE upscale_target >= 3840;
```

7.4 Access from Python

```
import db
rows = db.fetch(limit=10, search="cabin")
one = db.get(4)
totals = db.stats()
```

8. Performance

All figures were measured on the development machine (RTX 3090, 24 GB, driver 610.88, torch 2.14.1 with CUDA 12.6) and taken from generations.db.

8.1 Generation (native size)

Size	Steps	Inference	Steps/s	Peak VRAM	Temp after	Power after
1024x1024	15	4.7 to 4.9 s	3.1 to 3.2	10.5 GB	55 to 56 C	377 to 379 W
1024x1024	30	9.7 s	3.1	10.5 GB	-	-
1344x768	40	12.8 to 13.5 s	3.0 to 3.1	10.4 GB	52 to 54 C	349 to 357 W
768x1344	40	11.2 to 11.4 s	3.5 to 3.6	10.5 GB	55 to 57 C	382 to 385 W

Throughput is close to 3.1 steps per second at 1 megapixel regardless of aspect ratio. Inference time is therefore almost exactly proportional to the step count: about 0.32 s per step.

8.2 Upscaling

From	To	Passes	Time	Output file
1344x768	3840x2194 (4K)	1	3.4 s	9.9 MB
768x1344	2194x3840 (4K)	1	2.9 s	7.9 MB
768x1344	4388x7680 (8K)	2	8.4 to 8.5 s	30.5 MB

A standalone first call measured 9.4 s for the 4K case because it included loading the weights; inside the app the weights are already resident.

8.3 End-to-end

Request	Total wall time (API call to response)
16:9, 40 steps, 4K	20.6 s, of which 13.5 s inference, 3.4 s upscale, remainder PNG encoding and transfer
1024x1024, 25 steps, no upscale	9.9 s

8.4 Memory

Stage	VRAM
Idle with SDXL loaded	about 7 GB allocated, 15.7 GB reserved by the allocator
Peak during 1 MP generation	10.4 to 10.5 GB
Upscaler weights	67 MB (half precision)
Peak during tiled upscale	no measurable increase over the generation peak

The 15.7 GB "used" figure that nvidia-smi reports while idle is allocator reservation, not live tensors; it is reused rather than growing.

8.5 Startup

Step	Time
First ever start (downloads 6.7 GB)	bandwidth dependent, several minutes
SDXL load from local cache	27 to 43 s (disk cache dependent)
Upscaler load	under 1 s

8.6 Aggregate over the session

Metric	Value
Images recorded	7
Total GPU inference time	71 s
Average inference	10.0 s
Upscaled images	4
Total upscale time	23.2 s
Hottest reading	57 C
Peak board power	385 W of a 390 W limit

8.7 Tuning levers

- Steps: the dominant cost. 15 steps gives usable drafts in under 5 s; 40 steps is the quality default for 4K.
- Count: images in a batch are generated sequentially, so time scales linearly.
- 8K: costs about 5 s more than 4K and produces 30 MB files; the browser preview is noticeably slower to load.
- Cards with less VRAM: generate.py enables CPU offload below 12 GB, which roughly doubles inference time but keeps the pipeline working.

9. Operations and Troubleshooting

9.1 Starting and stopping

Action	How
Start	Double-click run-app.bat, or `.\.venv\Scripts\python.exe app.py` in the project folder
Confirm ready	Console shows "Model ready in N s", "Upscaler ready", "Running on local URL: http://127.0.0.1:7860"
Stop	Ctrl+C in the console, or close the window
Change port	Edit the server_port value in the last line of app.py
Expose on the LAN	Change server_name to "0.0.0.0" in app.py. There is no authentication, so only do this on a trusted network

9.2 Logs

The app prints one line per generated image to the console, for example:

```
#4 saved 20261022-005957_seed42.png  1344x768  13.527s  peak 10.46 GB  52.0 C  -> 3840x2194 in 3.38s
```

To keep a log file, redirect: `.\.venv\Scripts\python.exe -u app.py > app.log 2>&1`.

9.3 Known issues encountered during development and their fixes

Symptom	Cause	Fix applied
AttributeError: 'StableDiffusionXLPipeline' object has no attribute 'enable_vae_slicing'	diffusers 0.41 removed the pipeline-level helper	Call pipe.vae.enable_slicing() instead
Parser ignored "seed 42" and misread "21:9 steps 45" as 9 steps	Directive removal used offsets from a stale lowercase copy of the prompt	Rewrote removal to search and cut the live prompt string each time
Regex word boundaries silently vanished	A patch script wrote "\b" through a normal Python string, turning it into a backspace character	Restored with a byte-level replace; patches now use raw strings or the Write tool
UnicodeEncodeError: 'charmap' codec can't encode character	Windows console code page cp1252 cannot print the arrow and dot characters in reports	Set PYTHONIOENCODING=utf-8 when printing reports from scripts; the browser is unaffected
torchvision installed as the CPU build	pip pulled torchvision from PyPI as a dependency of spandrel before the CUDA index was consulted	Reinstalled with --force-reinstall from the cu126 index; requirements.txt documents the order
Warning: NumPy array is not writable	Tiles were passed as read-only views	Copy the array before converting to a tensor
Server died when a wait loop was cancelled	The server was a child of the same shell command	Launch the server as its own background process

9.4 Troubleshooting guide

Problem	Check	Resolution
"CUDA GPU not available" at startup	`python -c "import torch; print(torch.version.cuda)"` prints None	Reinstall torch from the cu126 index inside the virtual environment
Port 7860 already in use	Task Manager shows a python.exe from a previous run	End that process, or change the port
Out of memory during generation	Other GPU programs running; or a card under 12 GB	Close other GPU apps; use generate.py which offloads to CPU on small cards
First start hangs on "Loading"	Model download in progress	Wait; check %USERPROFILE%\.cache\huggingface\hub growth
Telemetry fields show None	nvidia-smi not on PATH	Reinstall the NVIDIA driver, or add C:\Windows\System32 to PATH
Upscaler weights missing	FileNotFoundError naming models/RealESRGAN_x4plus.pth	Download from the Real-ESRGAN v0.1.0 release into models/
History tab empty	New database, or generations.db moved	Generate an image; or restore the file next to db.py
Browser slow to show an 8K result	30 MB PNG transfer	Expected; open the file from outputs/ directly

9.5 Maintenance

- Outputs grow by 1.5 MB per native image and 8 to 31 MB per upscaled image. Prune outputs/ periodically; the database keeps the path and the History tab will show "not found" for deleted files.
- Back up outputs/ and generations.db together so paths stay valid.
- To reset history, stop the app and delete generations.db; it is recreated empty on next start.
- To update dependencies, upgrade inside the virtual environment and rerun the parser self-test and one generation. Pin versions in requirements.txt if a breaking change is found.

9.6 Security notes

- The server listens on localhost only by default and has no authentication.
- No telemetry or prompts are sent anywhere; the only network access is the one-time model download from Hugging Face.
- Prompts are stored verbatim in generations.db. Treat the file with the same care as the images.

10. Change Log and Roadmap

10.1 Change log

Increment 1: command-line generator (21 Oct 2026)

- Created the project folder, virtual environment, and requirements.txt.
- Installed torch 2.14.1+cu126, diffusers 0.41, transformers, accelerate.
- Wrote generate.py with prompt, negative, steps, guidance, size, seed, count and model arguments.
- First image: 1024x1024 in 9.7 s, peak VRAM 10.5 GB.
- Fixed: VAE slicing call renamed in diffusers 0.41.

Increment 2: web interface

- Added app.py using Gradio 6.30 with prompt, negative prompt, style and size dropdowns, step and guidance sliders, seed and count, a gallery and an info box.
- Model loaded once at startup and kept resident.
- Added run-app.bat and README.

Increment 3: prompt-driven controls

- Removed style, size, steps and guidance controls.
- Added prompt_parser.py: ratios, exact sizes, orientation words, quality and fidelity words, twelve style presets, count, seed, steps and guidance overrides.
- Added the live "Will generate" interpretation line and a collapsible help table.
- Fixed: stale-offset bug in directive removal; backspace characters introduced by a patch script.

Increment 4: telemetry and history

- Added gpu_info.py (static facts from torch, live readings from nvidia-smi).
- Added db.py with a 45-column SQLite table, insert, fetch, search, get, stats and a CLI viewer.
- app.py now times each stage, samples the GPU before and after, records every image, and renders a details report.
- Added the History tab: aggregate line, search, limit, table, row click to image and details.

Increment 5: real 4K and 8K output

- Added upscaler.py: Real-ESRGAN x4plus through spandrel, half precision, 512 px tiles with 32 px overlap, multi-pass to target, Lanczos finish.
- Parser: 2k, 4k, uhd, 8k words and oversize exact resolutions set an upscale target; exact sizes are centre-cropped.
- Database: eight upscale columns added with automatic migration.
- UI: upscaled image shown in the gallery and History; details report gains an Upscale section; header shows the upscaler.
- Measured: 4K in 3.4 s, 8K in 8.5 s on the RTX 3090.
- Fixed: torchvision CPU build replaced with the CUDA build.

Documentation (22 Oct 2026)

- Added docs/ with the documentation as PDFs (one combined, one per section) and docs-src/ with the Markdown sources, figures and build script.

10.2 Roadmap

Priority	Item	Notes
High	SDXL refiner or a diffusion-based "hires fix" pass	Adds generated detail at 2x before Real-ESRGAN; better fine texture than super-resolution alone
High	Image-to-image and inpainting tabs	Same pipeline family; lets a chosen image be edited rather than regenerated
Medium	Model switcher in the prompt, e.g. "model: sdxl-turbo"	Turbo gives 1 s drafts at 4 steps; FLUX fits in 24 GB for higher quality
Medium	Export history to CSV and chart timings	pandas is already present
Medium	Thumbnail cache for the History grid	Avoids loading full PNGs when browsing
Low	Authentication and LAN access	Gradio supports basic auth; needed before exposing beyond localhost
Low	Package as a Windows service or Docker image	For always-on use
Low	Cloud deployment profile	Hugging Face Space with ZeroGPU for sharing, or a RunPod template for burst capacity

Appendix

A. Project file tree

```
ai-image-gen/
  app.py                 web application (Gradio)
  prompt_parser.py       prompt -> settings
  upscaler.py            Real-ESRGAN tiled upscaling
  gpu_info.py            GPU facts and live telemetry
  db.py                  SQLite store and CLI viewer
  generate.py            command-line generator
  requirements.txt
  run-app.bat
  README.md
  generations.db         created on first generation
  models/
    RealESRGAN_x4plus.pth
  outputs/               generated PNG files
  docs/
    00-...-Complete-Technical-Documentation.pdf
    01-Project-Overview.pdf ... 11-Appendix.pdf   one PDF per section
  docs-src/
    01-overview.md ... 11-appendix.md   editable sources
    images/
    build_pdf.py
  .venv/                 virtual environment (not for backup)
```

B. requirements.txt

```
# Install PyTorch with CUDA first (RTX 3090 / any recent NVIDIA card):
#   pip install torch --index-url https://download.pytorch.org/whl/cu126
# Then:
#   pip install -r requirements.txt
diffusers>=0.31
transformers>=4.45
accelerate>=1.0
safetensors
gradio>=5.0
spandrel>=0.4
# torchvision: pip install torchvision --index-url https://download.pytorch.org/whl/cu126
```

C. Command cheat sheet

All commands run from the project folder in PowerShell.

Task	Command
Start the app	`.\.venv\Scripts\python.exe app.py`
Generate from CLI	`.\.venv\Scripts\python.exe generate.py "prompt" --seed 42 --count 2`
Upscale a file	`.\.venv\Scripts\python.exe upscaler.py outputs\x.png 3840`
List history	`.\.venv\Scripts\python.exe db.py`
One record	`.\.venv\Scripts\python.exe db.py 4`
Statistics	`.\.venv\Scripts\python.exe db.py stats`
Parser self-test	`.\.venv\Scripts\python.exe prompt_parser.py`
GPU telemetry dump	`.\.venv\Scripts\python.exe gpu_info.py`
Rebuild the PDFs	`.\.venv\Scripts\python.exe docs-src\build_pdf.py`
Check CUDA	`.\.venv\Scripts\python.exe -c "import torch; print(torch.cuda.is_available())"`

D. Programmatic access to the running app

```
from gradio_client import Client
c = Client("http://127.0.0.1:7860")
images, report = c.predict("a lighthouse, 16:9, 4k, seed 42", "", api_name="/generate")
table, stats = c.predict("lighthouse", 50, api_name="/refresh_history")
```

E. Licences

Component	Licence
Stable Diffusion XL base 1.0	CreativeML Open RAIL++-M
Real-ESRGAN weights and code	BSD 3-Clause
diffusers, transformers, accelerate	Apache 2.0
Gradio	Apache 2.0
spandrel	MIT
PyTorch	BSD-style
Project source code	Owned by Frazile AI